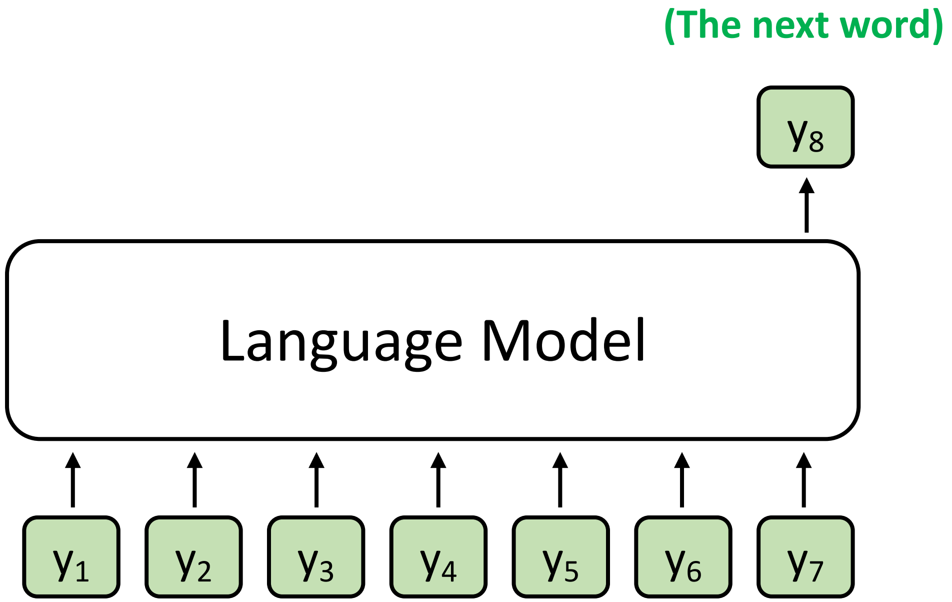
# Natural Language Processing

GPT-2 and T5 Tutorial 2024/11/05

# What you will learn in this tutorial

- Training and evaluating GPT-2 / T5 on abstractive summarization
  - Dataset: Chinese abstractive summarization
  - Objective: Cross-entropy
  - Main packages: PyTorch, Hugging Face, ROUGE

# Language Model




$$P(y_t | y_1, y_2, \ldots, y_{t-1})$$

基於過去文字的機率值，預測下一個會出現的字

- A model that assigns probabilities to upcoming words is called **a language model**.
- The task involving predictions of upcoming words is **language modeling**.

建模過程

# Why do we need to learn GPT-2?

- GPT-2 is the last open-source model in the GPT series from OpenAI.
- Language generation is hard, and GPT-2 is a good start point.
- GPT-2 is not big. It is feasible for low-budget machines.
  - GPT-2 (12-layer; 124M), GPT-2-medium (24-layer; 345M), GPT-2-large (36-layer; 762M), GPT-2-xl (48-layer; 1.5B)

# Introduction to Text Summarization

文本摘要任務

- Extractive summarization
  - Generate a short text summary for a document by selecting salient sentences in the documents. 將文本中的重要句子產出>>產出的內容會在資料範圍內
- Abstractive summarization
  - Generate novel words and phrases not featured in the source text.

消化完文本之後，產出句子>>產出的內容在資料範圍外


**Example:**
The Queen's Birthday holiday road toll stands at zero for the first time since records began.

**Ext summary:** The Queen's holiday road toll stands at zero.
**Abs summary:** The Queen's holiday slashes road toll.

# Task: Chinese Abstractive Summarization

- Dataset: LCSTS (A Large-Scale Chinese Short Text Summarization Dataset)[1]

**Short Text:** 水利部水资源司司长陈明忠今日在新闻发布会上透露，根据刚刚完成的水资源管理制度的考核，有部分省接近了红线的指标，有部分省超过红线的指标。在一些超过红线的地方，将对一些取用水项目进行区域的限批，严格地进行水资源论证和取水许可的批准。

**Summarization:** 部分省超过年度用水红线指标 取水项目将被限批

Hu, Baotian, Qingcai Chen, and Fangze Zhu. "LCSTS: A Large Scale Chinese Short Text Summarization Dataset." Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing. 2015.

Hugging Face | Search models, datasets, users... | Models | Datasets | Spaces | Posts | Docs | Pricing | Log In | Sign Up

# Documentations

https://huggingface.co/docs

Search across all docs

- **Transformers**
  State-of-the-art ML for Pytorch, TensorFlow, and JAX.
- **Diffusers**
  State-of-the-art diffusion models for image and audio generation in PyTorch.
- **Hub**
  Host Git-based models, datasets and Spaces on the Hugging Face Hub.
- **Datasets**
  Access and share datasets for computer vision, audio, and NLP tasks.
- **Gradio**
  Build machine learning demos and other web apps, in just a few lines of Python.
- **Hub Python Library**
  Client library for the HF Hub: manage repositories from your Python runtime.
- **Huggingface.js**
  A collection of JS libraries to interact with Hugging Face, with TS types included.
- **Transformers.js**
  Community library to run pretrained models from Transformers in your browser.
- **Inference API (serverless)**
  Experiment with over 200k models easily using the serverless tier of Inference Endpoints.
- **Inference Endpoints (dedicated)**
  Easily deploy models to production on dedicated, fully managed infrastructure.
- **PEFT**
  Parameter efficient finetuning methods for large models.

# Installation

Basically, you need to install PyTorch first before you install transformers.

(Recommended)

```
pip install transformers
```

(If you want to try new things)

```
git clone
https://github.com/huggingface/transformers.git
cd transformers
pip install -e .
```

# Packages required in this tutorial

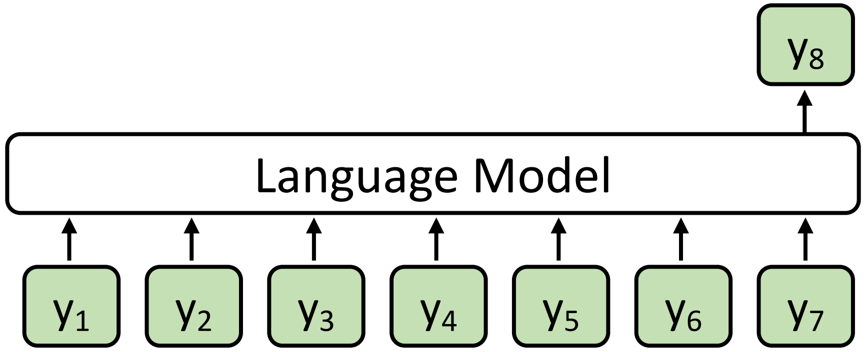
We need the following packages for this tutorial. On Colab, you may not need to re-install these packages for they may have already been installed.

```
! pip install torch==2.3.1 --index-url
https://download.pytorch.org/whl/cu121
!pip install transformers==4.37.0
!pip install datasets==2.21.0
!pip install accelerate==0.21.0
!pip install rouge==1.0.1
!pip install tqdm==4.66.5
!pip install jieba==0.42.1
```

# Language Model vs. Masked Language Model

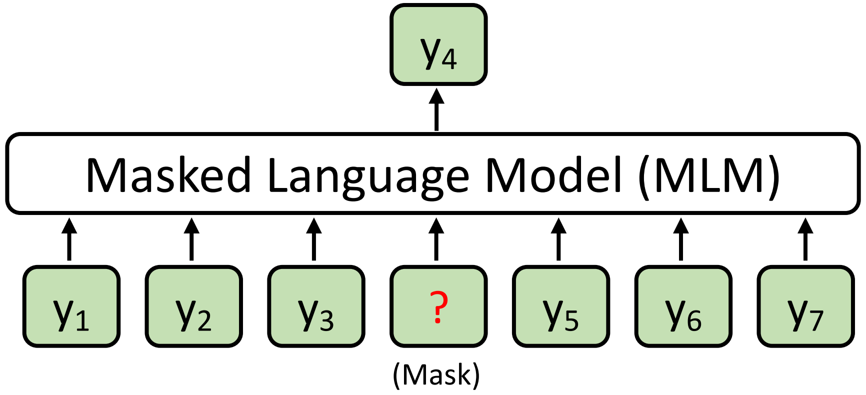
Casual Language Model
(E.g., GPT)

給於一串文字後 預測下一段文字




Masked Language Model
(E.g., BERT)

從上下文文本 預測在中間被masked掉的文字

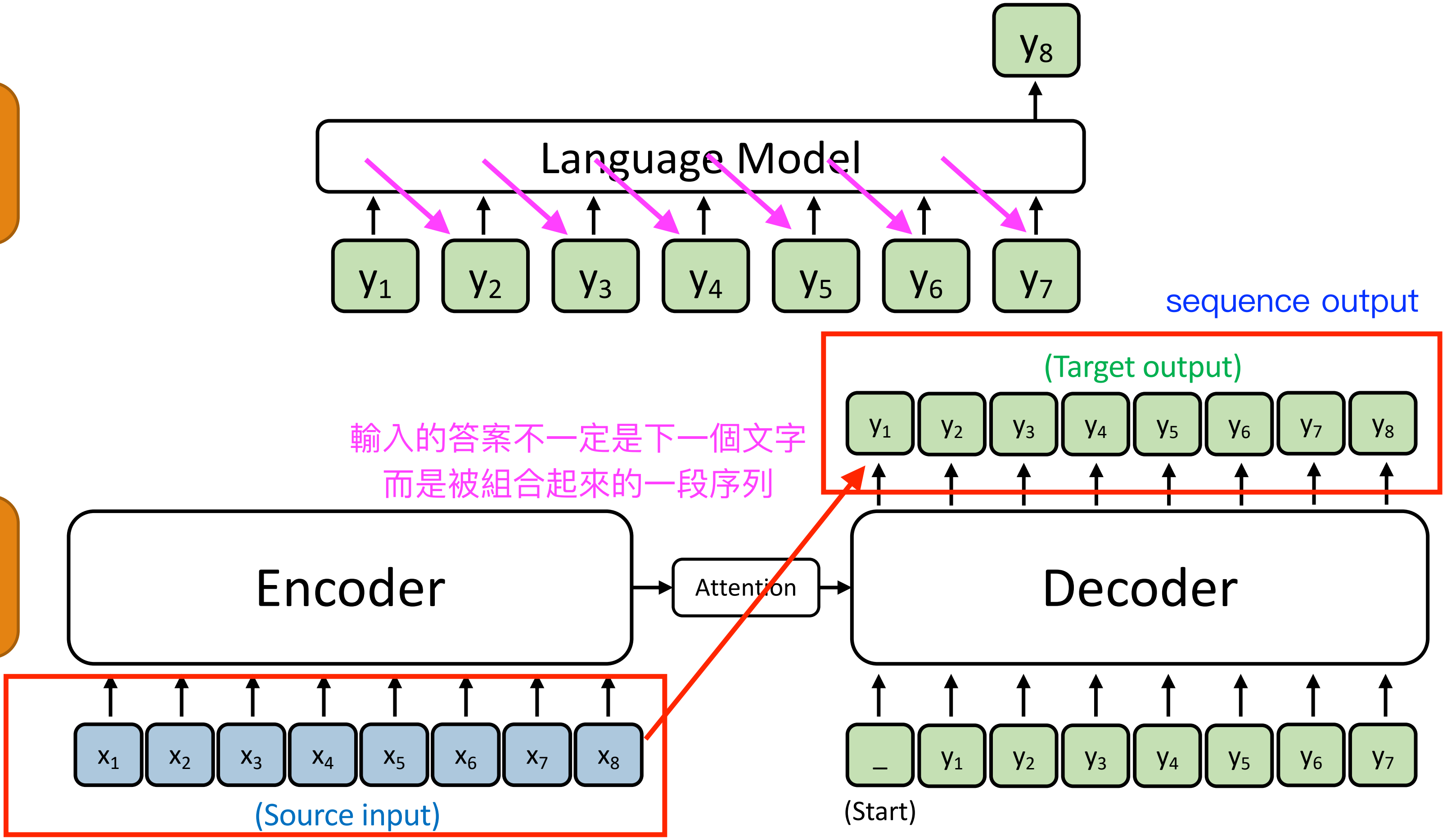


# Language Model vs. Seq2seq Model

Casual Language Model
(E.g., GPT)

僅encoder

Sequence to sequence Model
(E.g., T5; Vanilla Transformers)



# Load the LCSTS dataset

- [Hugging Face dataset link]()

```
raw_train = load_dataset(
    "hugcyp/LCSTS", split="train", cache_dir="./cache/"
).to_list()
raw_val = load_dataset(
    "hugcyp/LCSTS", split="validation", cache_dir="./cache/"
).to_list()
```

Sometimes checking the Hugging Face dataset is slow, it will be faster if we transform the dataset object into a list using `.to_list()`.

# Auto Classes (models, tokenizers)

- In many cases, the architecture you want to use **can be guessed** from the name or the path of the pretrained model you are supplying to the from_pretrained method.
- AutoClasses are here to do this job for you so that you automatically retrieve the relevant model given the name/path to the pretrained weights/config/vocabulary:

- Ex: model = AutoModel.from_pretrained('bert-base-cased') will create an instance of BertModel). 模型初始化過程>>取得pretrain model

https://huggingface.co/transformers/v3.0.2/model_doc/auto.html

# Generative Models in Auto Classes

| Model type | Classes |
| --- | --- |
| Casual Language Model (E.g., GPT) | AutoModelForCausalLM (link)<br>GPT2LMHeadModel (link) |
| Masked Language Model (E.g., BERT) | AutoModelForMaskedLM (link)<br>BertForMaskedLM (link) |
| Sequence to sequence Model (E.g., T5; Vanilla Transformers) | AutoModelForSeq2SeqLM (link)<br>T5ForConditionalGeneration (link) |

# Outline

- GPT-2 (native PyTorch)
- T5 (Hugging Face Dataset and Trainer)

# Import packages

```python
from transformers import AutoTokenizer
from transformers import GPT2LMHeadModel
from datasets import load_dataset
from tqdm import tqdm
import torch
from torch.utils.tensorboard import SummaryWriter
from rouge import Rouge
import jieba
```

# Load the Tokenizer and the Model

- Hugging Face model link

```
model_name = "uer/gpt2-chinese-cluecorpussmall"
tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    padding_side="left”,
)
tokenizer.add_special_tokens({"eos_token": "<|endoftext|>"})
model.resize_token_embeddings(len(tokenizer))
```

模型本身沒有eos_token/model生成的部分詞彙
=>通常eos_token出現之後，表示任務結束
=>因此此處多一個eos_token可以儲存暫時性的詞彙，以便最後會出一段句子

因為要儲存的詞彙量變多，所以要resize/擴增dictionary

Optional

模型應該關注在哪部分？

# Left padding

一個batch
=>如果生成的部分太短，會加上pad補足
可統一長度

句子過短，model會看到過多
padding，最後導致生成效果被影響

In text generation, sometimes we set padding_side='left', when loading tokenizer.

If padding is at the end of the prompt, new tokens may be generated after the padding, which is illogical.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| <s> | Recite | the | first | law | <s> | <pad> | <pad> | <pad> | 生成文字區 |
| <s> | How | are | you | <s> | <pad> | <pad> | <pad> | <pad> | |
| <s> | Who | is | the | first | president | of | U.S. | <s> | |

right–padding:pad集中在右半部

## Left-padding

Aligning the new tokens.

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| <pad> | <pad> | <pad> | <s> | Recite | the | first | law | <s> | 生成文字區 |
| <pad> | <pad> | <pad> | <pad> | <s> | How | are | you | <s> | |
| <s> | Who | is | the | first | president | of | U.S. | <s> | |

left–padding:pad集中在左半部，這樣可以讓文字生成區比較不會受干擾

# About Padding

- When fine-tuning GPT-2
  - Left padding + set [PAD] as -100 in labels (this tutorial)
  - Right padding + set [PAD] as -100 in labels
- When using GPT-2 for test (inference time)
  - Use test_batch_size as 1 (this tutorial) 一次測試只拿一個句子=>不需要padding
  - Left padding

# Set [PAD] as -100 in labels

https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py#L1264-L1267

```
r"""
labels (`torch.LongTensor` of shape `(batch_size, sequence_length)`, *optional*):
    Labels for language modeling. Note that the labels **are shifted** inside the model, i.e. you can set
    `labels = input_ids` Indices are selected in `[-100, 0, ..., config.vocab_size]` All labels set to `-100`
    are ignored (masked), the loss is only computed for labels in `[0, ..., config.vocab_size]`
"""
```

loss調整不會因為pad的部分被影響到，相當於被masked掉

# Load the Tokenizer and the Model

- [Hugging Face model link]

```
model_name = "uer/gpt2-chinese-cluecorpussmall"
tokenizer = AutoTokenizer.from_pretrained(
    model_name,
    padding_side="left”,
)
tokenizer.add_special_tokens({"eos_token": "<|endoftext|>"})
model.resize_token_embeddings(len(tokenizer))
```

Increasing the size will add newly initialized vectors at the end.
Reducing the size will remove vectors from the end. (link)

# Create the PyTorch Dataset

- You can use the Hugging Face dataset without building a PyTorch dataset class.
- However, language generation is really complicated. We suggest building your first code with native PyTorch.

```python
class LCSTSDataset(torch.utils.data.Dataset):
    def __init__(self, raw_data) -> None:
        super().__init__()
        self.data = raw_data
        self.token_replacement = [
            ["：", ":"],
            ["，", ","],
            ["“", '"'],
            ["”", '"'],
            ["？", "?"],
            ["……", "..."],
            ["！", "!"],
        ]

    def __getitem__(self, index):
        d = self.data[index]
        # Substitute some full-width punctuations with half-width ones
        for k in d:
            for tok in self.token_replacement:
                d[k] = d[k].replace(tok[0], tok[1])
        return d

    def __len__(self):
        return len(self.data)
```

token–replacement:
將不存在字典裡面的字/token內容汰換掉
=> Ex. 全形符號換成半形符號
=>如果是找不到的符號會轉變成UNK

To prevent out-of-vocabulary tokens from being transformed into [UNK]

Run:

```python
train_set = LCSTSDataset(raw_train)
val_set = LCSTSDataset(raw_val)
```

# Create the PyTorch DataLoader for Data Batching

1. Input data for training (source + summary)
2. Labels (auto-regressive; thus, basically input_ids themselves)
   自己的train data–>自己預測出的結果–>作為正確答案
3. Input data for predictions (source only)

1.input data
=>model走 auto regression形式(每次只預測下一個字)
=>source: 把所有的training data讀完
=>summary: 把所有source讀完之後生成的文字
2.Labels
=>auto–regressives–>就是label本身
3.Predictions
=>model train完之後，要檢視其生成的能力
=>所以只看source

# Create the PyTorch DataLoader for Data Batching: 1. Input Data for Training

```python
def collate_fn(batch):
    complete_text = [
        f"[CLS]{example['text']}[SEP]{example['summary']}<|endoftext|>"
        for example in batch
    ]
    complete_text = tokenizer.batch_encode_plus(
        complete_text,
        padding=True,
        truncation=True,
        return_tensors="pt",
        add_special_tokens=False,
    )
```

You can omit the [CLS] token before fine-tuning!

[CLS] | Source article | [SEP] | Summary | <|endoftext|>

model只要看到sep
就會開始進行生成摘要

eos_token
(看到這個token之後，就不再生成)

# uer/gpt2-chinese-cluecorpussmall uses BertTokenizer

https://huggingface.co/uer/gpt2-chinese-cluecorpussmall/blob/main/config.json#L26

```
  "task_specific_params": {
    "text-generation": {
      "do_sample": true,
      "max_length": 320
    }
  },
  "tokenizer_class": "BertTokenizer",
  "vocab_size": 21128
```

https://huggingface.co/ckiplab/gpt2-base-chinese/blob/main/config.json#L32

```
  "task_specific_params": {
    "text-generation": {
      "do_sample": true,
      "max_length": 50
    }
  },
  "tokenizer_class": "BertTokenizerFast",
  "vocab_size": 21128
```

# English GPT-2 for Text Summarization

> To induce summarization behavior, we add the text TL;DR: after the article [2]

token,適用在英文

TL;DR

Ideally, the model may learn [SEP] as a signal to perform summarization.

[CLS] Source article [SEP] Summary <|endoftext|>

(You can try it.)

Ideally, the model may learn <|endoftext|> as a signal to end generation.

[2] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. OpenAI blog, 1(8), 9.

# Create the PyTorch DataLoader for Data Batching: 2. Labels (auto-regressive)

(We are still at the `collate_fn` function.)

```python
    # Set label padding tokens to -100 for loss masking
    labels = torch.where(
        condition=complete_text.input_ids != tokenizer.pad_token_id,
        input=complete_text.input_ids,
        other=-100,
    )
    complete_text["labels"] = labels
    complete_text = {k: complete_text[k].to(device) for k in complete_text}
```

This is sentence A.

This is is is sentence B.

會被移除

| | | | | | | |
|---|---|---|---|---|---|---|
| | [PAD] | [PAD] | this | is | sentence | a |
| labels | -100 | -100 | 1212 | 318 | 6827 | 317 |
| | this | is | is | is | sentence | b |
| labels | 1212 | 318 | 318 | 318 | 6827 | 347 |

# GPT2LMHeadModel shifts logits and labels

https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py#L1300-L1301

```
1296         if labels is not None:
1297             # move labels to correct device to enable model parallelism
1298             labels = labels.to(lm_logits.device)
1299             # Shift so that tokens < n predict n
1300             shift_logits = lm_logits[..., :-1, :].contiguous() 最後面少一格    =>對齊input文字可以
1301             shift_labels = labels[..., 1:].contiguous()  最後面多一格         預測下一個label
1302             # Flatten the tokens
1303             loss_fct = CrossEntropyLoss()
1304             loss = loss_fct(shift_logits.view(-1, shift_logits.size(-1)), shift_labels.view(-1))
```

# Create the PyTorch DataLoader for Data Batching: 3. Input Data for Predictions

(We are still at the `collate_fn` function.)

We will use `infer_text` during evaluations.

```python
    infer_text = [example["text"] for example in batch]
    infer_text = tokenizer.batch_encode_plus(
        infer_text,
        padding=True,
        truncation=True,
        return_tensors="pt",
    )
    infer_text = {k: infer_text[k].to(device) for k in infer_text}
    return complete_text, infer_text
```

If you omit the [CLS] token during fine-tuning, then you don’t need to add [CLS] before evaluations!

[CLS] | Source article | [SEP] | ~~Summary~~ | ~~<|endoftext|>~~

# Create the PyTorch DataLoader for Data Batching

```
train_loader = torch.utils.data.DataLoader(
    train_set,
    batch_size=TRAIN_BATCH_SIZE,
    shuffle=True,
    collate_fn=collate_fn,
)
val_loader = torch.utils.data.DataLoader(
    val_set,
    batch_size=VAL_BATCH_SIZE,
    shuffle=False,
    collate_fn=collate_fn,
)
```

資料的順序性可能會被model 學習
=>shuffle打亂順序性之後，可以避免學習到資料順序，更著重在資料本身學習

Prevent a model from overfitting on data order

驗證資料shuffle沒差

We don’t need to use the test set because there is no public test labels (summaries).
See https://huggingface.co/datasets/hugcyp/LCSTS/viewer/default/test

# Set up Optimizer

```
optimizer = torch.optim.AdamW(model.parameters(), lr=LR)
```

- **optimizers** (`Tuple[torch.optim.Optimizer, torch.optim.lr_scheduler.LambdaLR]`, *optional*, defaults to `(None, None)`) — A tuple containing the optimizer and the scheduler to use. Will default to an instance of AdamW on your model and a scheduler given by get_linear_schedule_with_warmup() controlled by `args`.

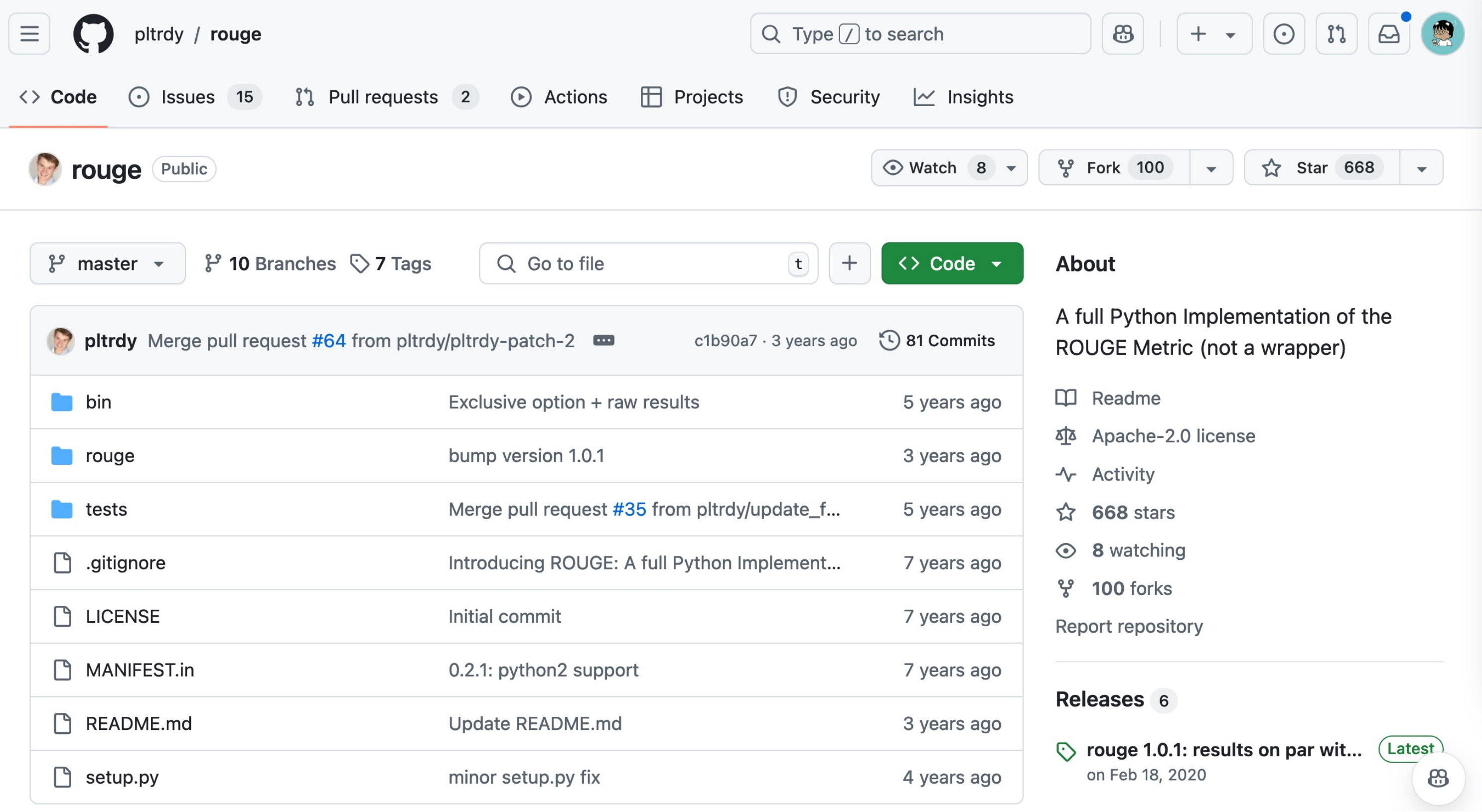
Trainer

- Trainer
- Seq2SeqTrainer
- TrainingArguments
- Seq2SeqTrainingArguments

https://pytorch.org/docs/stable/generated/torch.optim.AdamW.html
https://huggingface.co/docs/transformers/main_classes/trainer

# Set up Evaluation Metric

```
rouge_metric = Rouge()
```




https://github.com/pltrdy/rouge

# ROUGE Score

一種評估標準
=>主要在計算overlapping程度

模型所預測的目標=>recall越高越好

- ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
- Mainly for text summarization

計算prediction和summary有多少重疊
=> 越高,表示refernce和summary越多一樣的
=>模型表現更好

- Metric Input: Summary (prediction), Reference (gold summary)
- Common metrics: ROUGE-1, ROUGE-2, ROUGE-L

single–gram/bi–gram計算

最長的refernce和summary的匹配程度

  - L: Longest common subsequence
- Please note that current papers calculate ROUGE-F as default!!!
  - In other words, ROUGE-1F, ROUGE-2F, ROUGE-LF

Lin, Chin-Yew. "Rouge: A package for automatic evaluation of summaries." Text summarization branches out. 2004.

# ROUGE-1 Example

predictions = ["The", "cat", "sat", "on", "the", "mat"]

references = ["A", "cat", "was", "sitting", "on", "the", "mat"]

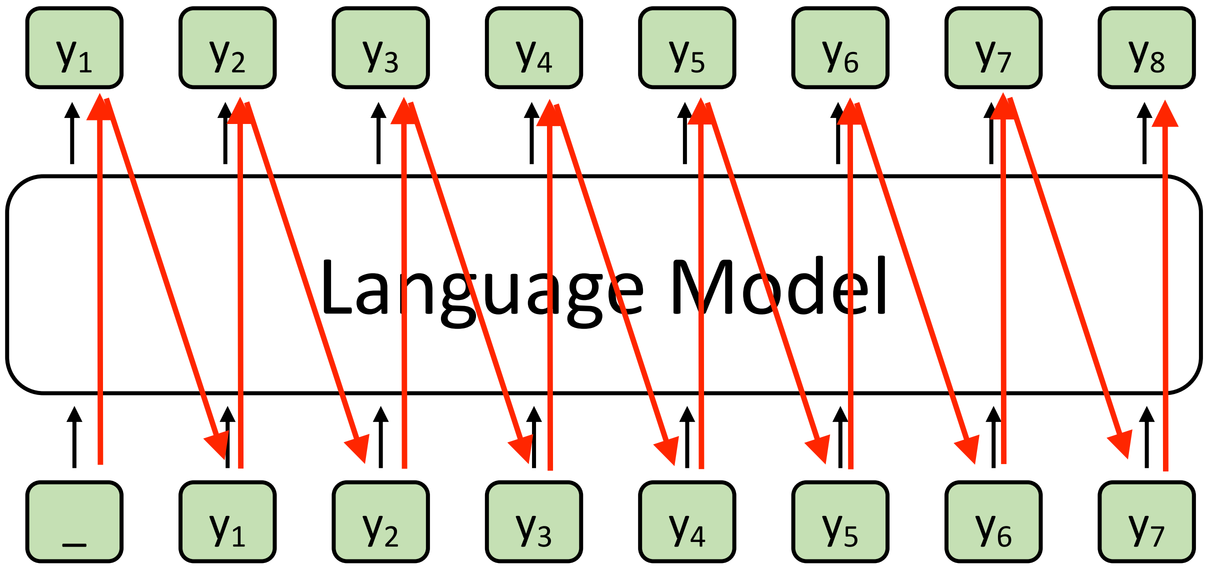
ROUGE-1 recall = Number of matching unigrams / Number of unigrams in the reference = 4/7

ROUGE-1 precision = Number of matching unigrams / Number of unigrams in the machine-generated summary = 4/6

ROUGE-1 F1-score = Harmonic mean of the precision and the recall = 2 * 4/7 * 4/6 / (4/7 + 4/6)

# Generating step-by-step

Casual Language Model
(E.g., GPT)




- Training time: update the model through hidden states 不會移交到decode，這樣才可以延續training達到更好的訓練效果
- Test time: perform decoding for each step 需要每次test玩都decode一次，進行驗證後優化

# Decoding strategies

- Popular decoding strategies:
  - Greedy decoding
  - Beam search
  - Top-k sampling
  - Top-p sampling

https://huggingface.co/blog/how-to-generate

# How to let a model generate step-by-step?

- We can use model.generate()
- There are two main advantages of model.generate():
  1. We don’t need to write a decoding loop on our own.
  2. We don’t need to implement decoding strategies by ourselves.

https://huggingface.co/blog/how-to-generate

# Perform Evaluations

Input: `[CLS]` `Source article` `[SEP]`

在evaluation/評估過程中，因此不需要summary

Expected output of `model.generate()`:

model generate()產出物通常包含input +summary/預測答案的部分

`[CLS]` `Source article` `[SEP]` `Summary` `<|endoftext|>`

Gold: `Summary`

# Perform Evaluations

## Output Part

```python
def do_evaluate(
    tokenizer,
    model,
    validation_loader,
    rouge_metric,
    inner_check=False,
):
    pbar = tqdm(validation_loader)
    pbar.set_description(f"Evaluating")

    predictions = []
    references = []
    count = 0
    for ground_truth, inputs in pbar:
        output = [
            s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]
            for s in tokenizer.batch_decode(
                model.generate(
                    **inputs,
                    max_new_tokens=200,
                    pad_token_id=tokenizer.eos_token_id,
                )
            )
        ]
```

[CLS]+ source article+ [SEP]
=>不含summary(正確答案)

endoftext取零相當於summary

decoder過程
=>每個token都會放空白鍵
=>為了不影響decode完的呈現效果
因此先把空白鍵取代

Expected output of `model.generate()`:

[CLS] | Source article | [SEP] | Summary | <|endoftext|>

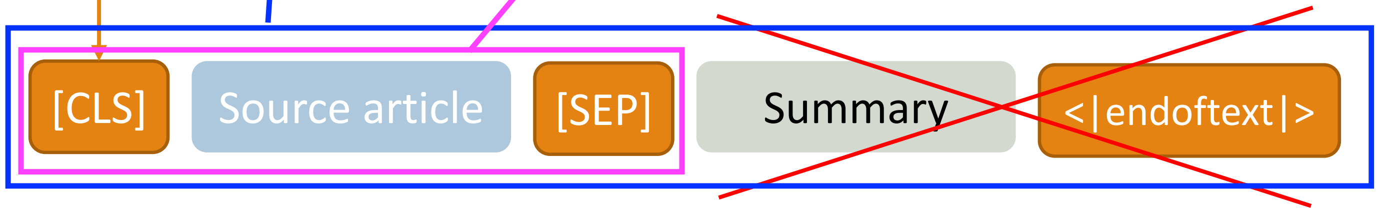
# Create the PyTorch DataLoader for Data Batching: 3. Input Data for Predictions

(We are still at the `collate_fn` function.)

We will use `infer_text` during evaluations.

```
    infer_text = [example["text"] for example in batch]
    infer_text = tokenizer.batch_encode_plus(
        infer_text,
        padding=True,
        truncation=True,
        return_tensors="pt",
    )
    infer_text = {k: infer_text[k].to(device) for k in infer_text}
    return complete_text, infer_text
```

If you omit the [CLS] token during fine-tuning, then you don’t need to add [CLS] before evaluations!



# Perform Evaluations

## Output Part

```python
def do_evaluate(
    tokenizer,
    model,
    validation_loader,
    rouge_metric,
    inner_check=False,
):
    pbar = tqdm(validation_loader)
    pbar.set_description(f"Evaluating")

    predictions = []
    references = []
    count = 0
    for ground_truth, inputs in pbar:
        output = [
            s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]
            for s in tokenizer.batch_decode(
                model.generate(
                    **inputs,
                    max_new_tokens=200,
                    pad_token_id=tokenizer.eos_token_id,
                )
            )
        ]
```

If you don’t set max_new_tokens, Hugging Face will also count the input tokens!

因為model generate通常會包含input token
所以如果不設定max_new_token的話，會計算到input token
=>若input token就已經超過200的話，就沒有辦法生成summary了

# Perform Evaluations

# Output Part

```python
def do_evaluate(
    tokenizer,
    model,
    validation_loader,
    rouge_metric,
    inner_check=False,
):
    pbar = tqdm(validation_loader)
    pbar.set_description(f"Evaluating")

    predictions = []
    references = []
    count = 0
    for ground_truth, inputs in pbar:
        output = [
            s.split("[SEP]")[1].replace(" ", "").split("<|endoftext|>")[0]
            for s in tokenizer.batch_decode(
                model.generate(
                    **inputs,
                    max_new_tokens=200,
                    pad_token_id=tokenizer.eos_token_id,
                )
            )
        ]
```

For each batch, first finished sentences should have <|endoftext|> rather than [PAD] at the end.

句子的結尾只能加上eos token，不可加上pad

More details:
https://github.com/huggingface/transformers/blob/b880508440f43f80e35a78ccd2a32f3bde91cb23/src/transformers/generation_utils.py#L1248-L1251

# Perform Evaluations

# Target and Scoring Parts

```python
        targets = [
            s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")
            for s in tokenizer.batch_decode(ground_truth["input_ids"])
        ]
        assert len(output) == len(targets)
        output = [" "] if output == [""] else output
        predictions.extend([" ".join(jieba.lcut(o)) for o in output])
        references.extend([" ".join(jieba.lcut(t)) for t in targets])
        count += 1
        if count > 100 and inner_check:
            break

    score = rouge_metric.get_scores(predictions, references, avg=True)
    if inner_check:
        print("Validation using 100 examples: ", score)
    else:
        print(score)

    return score, predictions, references
```

ground_truth:

target

[CLS] | Source article | [SEP] | Summary | <|endoftext|>

# Perform Evaluations

# Target and Scoring Parts

算積分的方式
=>reference和predict的部分有多少重疊

```
        targets = [
            s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")
            for s in tokenizer.batch_decode(ground_truth["input_ids"])
        ]
        assert len(output) == len(targets)
        output = [" "] if output == [""] else output
        predictions.extend([" ".join(jieba.lcut(o)) for o in output])
        references.extend([" ".join(jieba.lcut(t)) for t in targets])
        count += 1
        if count > 100 and inner_check:
            break

    score = rouge_metric.get_scores(predictions, references, avg=True)
    if inner_check:
        print("Validation using 100 examples: ", score)
    else:
        print(score)

    return score, predictions, references
```

training過程中，固定每100個就
先用valid dataset做驗證

把連續的詞轉換成實際文字的形式(包含段詞、空白等等)
=>jieba: 把一串string作段詞>轉變成list形式(lcut)>前面加上
“ ”空白鍵>模擬成真實句子的格式

Character-level bi-gram: (「看」、「電」)(「電」、「視」)

Word-level bi-gram: (「看」、「電視」)

rouge metric用word-level算比較貼近中文
文句應用=>用character-level算會比較高分
但是針對斷句就沒有學習到

# Perform Evaluations

# Target and Scoring Parts

```python
        targets = [
            s.split("[SEP]")[1].replace(" ", "").replace("<|endoftext|>", "")
            for s in tokenizer.batch_decode(ground_truth["input_ids"])
        ]
        assert len(output) == len(targets)
        output = [" "] if output == [""] else output
        predictions.extend([" ".join(jieba.lcut(o)) for o in output])
        references.extend([" ".join(jieba.lcut(t)) for t in targets])
        count += 1
        if count > 100 and inner_check:
            break

    score = rouge_metric.get_scores(predictions, references, avg=True)
    if inner_check:
        print("Validation using 100 examples: ", score)
    else:
        print(score)

    return score, predictions, references
```

avg=True: perform averaging for all the tested instances

# Training Loop

## Evaluating the subset of the validation data at each 1,000 steps

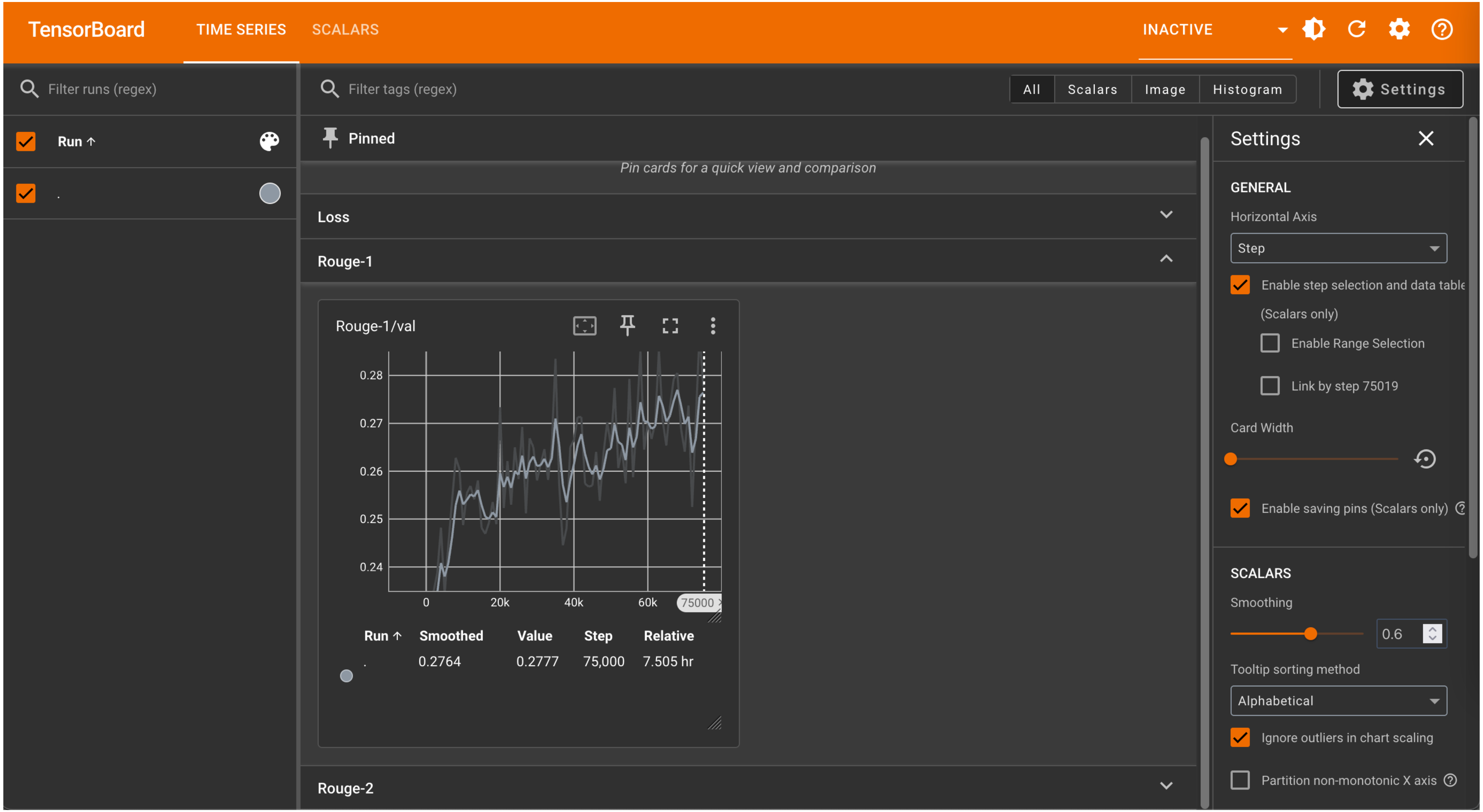
```python
step_i = 0
for epoch in range(NUM_EPOCHS):
    pbar = tqdm(train_loader)
    pbar.set_description(f"Training epoch [{epoch+1}/{NUM_EPOCHS}]")
    for inputs, _ in pbar:
        optimizer.zero_grad()
        loss = model(**inputs).loss
        loss.backward()
        optimizer.step()
        pbar.set_postfix(loss=loss.item())
        writer.add_scalar("Loss/train", loss.item(), step_i)

        if step_i % 1000 == 0 and step_i != 0:
            score, pres, refs = do_evaluate(
                tokenizer=tokenizer,
                model=model,
                validation_loader=val_loader,
                rouge_metric=rouge_metric,
                inner_check=True,
            )
            print(f"Rouge scores on step{step_i} of epoch {epoch}:", score)
            print("Predictions:", pres[:5])
            writer.add_scalar("Rouge-1/val", score["rouge-1"]["f"], step_i)
            writer.add_scalar("Rouge-2/val", score["rouge-2"]["f"], step_i)
        step_i += 1
```

將value寫到tensor flow

每1000個step就做一次validation評估

Tensor flow每次都會記錄loss、 rouge1、 rouge2的值


TensorBoard
TIME SERIES
SCALARS
INACTIVE
Filter runs (regex)
Filter tags (regex)
All
Scalars
Image
Histogram
Settings
Run ↑
.
Pinned
Pin cards for a quick view and comparison
Loss
Rouge-1
Rouge-1/val
0.28
0.27
0.26
0.25
0.24
0
20k
40k
60k
75000
Run ↑
Smoothed
Value
Step
Relative
.
0.2764
0.2777
75,000
7.505 hr
Rouge-2
Settings
GENERAL
Horizontal Axis
Step
Enable step selection and data table
(Scalars only)
Enable Range Selection
Link by step 75019
Card Width
Enable saving pins (Scalars only)
SCALARS
Smoothing
0.6
Tooltip sorting method
Alphabetical
Ignore outliers in chart scaling
Partition non-monotonic X axis

# Training Loop

Evaluating the full validation set at each end of epoch

```
score, pres, refs = do_evaluate(
    tokenizer=tokenizer,
    model=model,
    validation_loader=val_loader,
    rouge_metric=rouge_metric,
)
torch.save(model, f"{SAVED_DIR}/ep{epoch}.ckpt")
```

每訓練完一個epoch就留一個暫存檔

# Outline

- GPT-2 (native PyTorch)
- T5 (Hugging Face Dataset and Trainer)

# T5 [3]

功能性多
=>sequence to sequence代表模型

Raffel, Colin, et al. "Exploring the limits of transfer learning with a unified text-to-text transformer." *Journal of machine learning research* 21.140 (2020): 1-67.

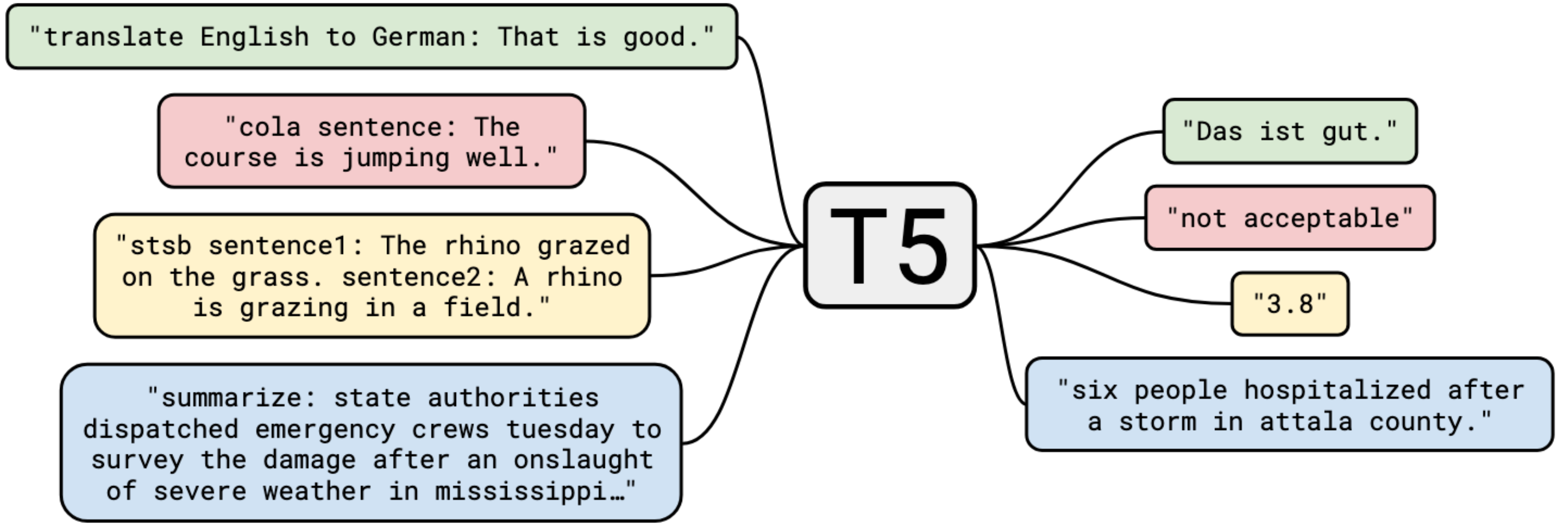
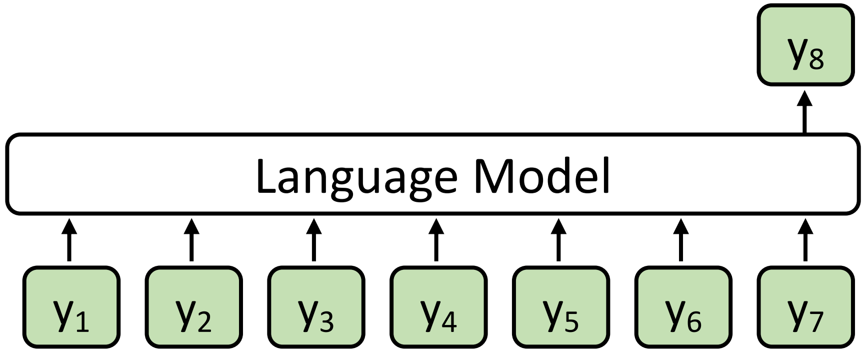



Figure 1: A diagram of our text-to-text framework. Every task we consider—including translation, question answering, and classification—is cast as feeding our model text as input and training it to generate some target text. This allows us to use the same model, loss function, hyperparameters, etc. across our diverse set of tasks. It also provides a standard testbed for the methods included in our empirical survey. "T5" refers to our model, which we dub the "**T**ext-**t**o-**T**ext **T**ransfer **T**ransformer".

# Language Model vs. Seq2seq Model

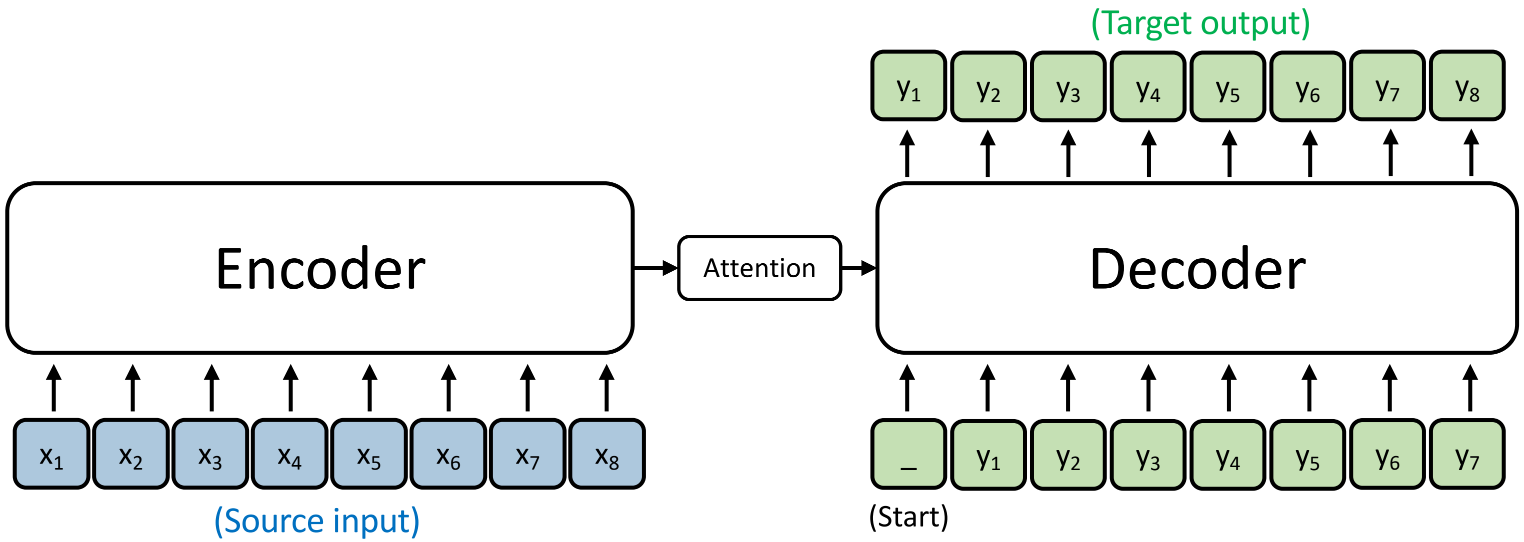
Casual Language Model
(E.g., GPT)

只預測下一個字




Sequence to sequence Model
(E.g., T5; Vanilla Transformers)

輸入source input/輸出target output



# Import packages

```
from transformers import AutoTokenizer
from transformers import AutoModelForSeq2SeqLM
from transformers import DataCollatorForSeq2Seq
from transformers import Seq2SeqTrainingArguments
from transformers import Seq2SeqTrainer
from datasets import load_dataset
from rouge import Rouge
import numpy as np
import pickle
import os
import jieba
```

# Load the LCSTS dataset

- [Hugging Face dataset link]

```
data_name = "hugcyp/LCSTS"
raw_train = load_dataset(data_name, split="train")
raw_valid = load_dataset(data_name, split="validation")
raw_small_valid = raw_valid.select(range(100))
```

只先取出前100做前處理

- raw_small_valid will be used for inner_check (validation).
- We will later process the dataset with the built-in map function, so we don't use `.to_list()` here.

# Load the Tokenizer and the Model

- Hugging Face model link

```
model_name = "google/mt5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
```

多語系的t5 small model

原本：需要將所有句子對齊後才生成，所以需要加上pad進行填空
現在：sequence to sequence不需要對齊，因此不需要pad

Q: Do we need left-padding?
A: No, because we are now using a **sequence-to-sequence model**, which will first compress input sequences though the encoder.

Q: Do we need to add [EOS] by ourselves?
A: No, **mT5** has </s> as the [EOS] token.

# Data pre-processing (inputs; source articles)

```python
token_replacement = [
    ["：", ":"],
    ["，", ","],
    ["“", '"'],
    ["”", '"'],
    ["？", "?"],
    ["……", "..."],
    ["！", "!"],
]
```

GPT–2:字回歸模型

```python
def replace_tokens(examples):
    for k in ["text", "summary"]:
        for i, _ in enumerate(examples[k]):
            for tok in token_replacement:
                examples[k][i] = examples[k][i].replace(tok[0], tok[1])
    return examples


def preprocess_function(examples):
    examples = replace_tokens(examples)
    model_inputs = tokenizer(examples["text"], padding=True, truncation=True)
    labels = tokenizer(
        text_target=examples["summary"],
        max_length=200,
        truncation=True,
    )
    model_inputs["labels"] = labels["input_ids"]

    return model_inputs
```

全半形符號代換

只能text進行encoding

# Data pre-processing (labels; summaries)

```
token_replacement = [
    ["：", ":"],
    ["，", ","],
    ["“", '"'],
    ["”", '"'],
    ["？", "?"],
    ["……", "..."],
    ["！", "!"]
]
```

```
def replace_tokens(examples):
    for k in ["text", "summary"]:
        for i, _ in enumerate(examples[k]):
            for tok in token_replacement:
                examples[k][i] = examples[k][i].replace(tok[0], tok[1])
    return examples


def preprocess_function(examples):
    examples = replace_tokens(examples)
    model_inputs = tokenizer(examples["text"], padding=True, truncation=True)
    labels = tokenizer(
        text_target=examples["summary"],
        max_length=200,
        truncation=True,
    )
    model_inputs["labels"] = labels["input_ids"]

    return model_inputs
```

與GPT最大的不同

=>label = input,有包含summary

# Data pre-processing (executions)

```
train = raw_train.map(preprocess_function, batched=True)
valid = raw_valid.map(preprocess_function, batched=True)
small_valid = raw_small_valid.map(preprocess_function, batched=True)
```

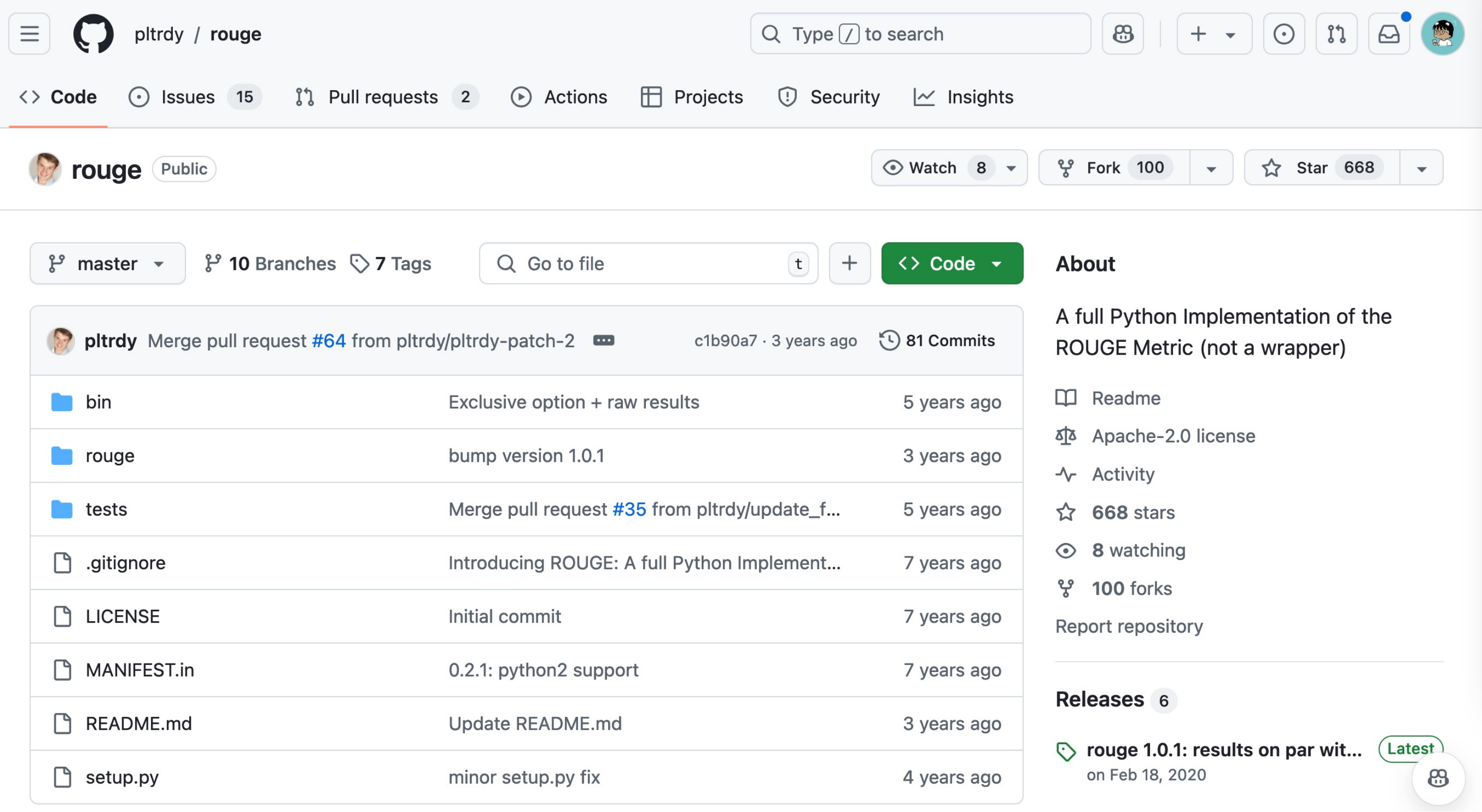
```
data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=model_name,
)
```

動態調整

DataCollatorForSeq2Seq dynamically pads batched data and transforms padded labels into -100.
The operation provided by this object does a similar job like collate_fn.

# Set up Evaluation Metric

```
rouge_metric = Rouge()
```




https://github.com/pltrdy/rouge

# Build `compute_metrics` for evaluations

```python
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    decoded_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)
    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    predictions = [" ".join(jieba.lcut(o)) for o in decoded_preds]
    references = [" ".join(jieba.lcut(t)) for t in decoded_labels]
    result = rouge_metric.get_scores(predictions, references, avg=True)
    score = {f"{rouge_i}_f": v["f"] for rouge_i, v in result.items()}
    prediction_lens = [
        np.count_nonzero(pred != tokenizer.pad_token_id) for pred in predictions
    ]
    score["gen_len"] = np.mean(prediction_lens)
    return {k: v for k, v in score.items()}
```

預測prediction,labels

把–100換回程
pad_id

-100 is transformed by DataCollatorForSeq2Seq.
Convert labels with -100 to pad_token_id for decoding.

# Setting up TrainingArguments ([link](link))

```python
training_args = Seq2SeqTrainingArguments(
    output_dir="./results/mt5",
    evaluation_strategy="steps",
    save_strategy="steps",
    eval_steps=1000,
    save_steps=10000,
    learning_rate=2e-5,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    weight_decay=0.01,
    save_total_limit=3,
    num_train_epochs=3,
    predict_with_generate=True,
    logging_dir=f"./logs/{model_prefix}",
    logging_steps=1,
    push_to_hub=False,
)
```

Default using Greedy search. Seq2seqTrainer Supports beam search only.

# Setting up Trainer (training part)

```python
trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=train,
    eval_dataset=small_valid,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
trainer.args._n_gpu = 1
trainer.train()
results = trainer.predict(valid)
for metric in ["1", "2", "l"]:
    rouge_item = f"test_rouge-{metric}"
    print(f"{rouge_item}: ", results.metrics[rouge_item])
```

# Setting up Trainer (evaluation part)

```python
trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=train,
    eval_dataset=small_valid,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
trainer.args._n_gpu = 1
trainer.train()
results = trainer.predict(valid)
for metric in ["1", "2", "l"]:
    rouge_item = f"test_rouge-{metric}"
    print(f"{rouge_item}: ", results.metrics[rouge_item])
```

# Thank you!